

Rapport de Machine Learning

Etude et Comparaison des Algorithmes de Machine Learning

Realise par :

Aziza Halmaouy

Encadre par :

Professeur Fahd Kalloubi

Module : Machine Learning

Annee universitaire : 2025 / 2026

Projet GitHub :

<https://github.com/aziza-halmaouy/ML-Projet>

Notebook Colab :

<https://colab.research.google.com/github/aziza-halmaouy/ML-Projet>

1. Introduction generale

Le Machine Learning est un domaine de l'intelligence artificielle qui permet de developper des modeles capables d'apprendre automatiquement a partir des donnees, sans etre explicitement programmes pour chaque tache. Il est aujourd'hui largement utilise dans divers secteurs tels que la sante, la finance, le transport et l'analyse comportementale.

On distingue principalement deux approches :

- **Apprentissage supervise (Supervised Learning)** : base sur des donnees etiquetees ou la variable cible est connue, permettant de faire des predictions et des classifications.
- **Apprentissage non supervise (Unsupervised Learning)** : base sur des donnees non etiquetees, utilise pour decouvrir des structures ou des regroupements caches dans les donnees.

Dans ce travail, nous avons etudie et compare plusieurs algorithmes de Machine Learning appliques a un dataset de chauffeurs issu du Chicago Open Data Portal. L'objectif est d'analyser les performances des modeles afin d'identifier les methodes les plus adaptees pour la classification, la prediction et le regroupement des donnees.

1.1 Description du dataset

Le dataset utilise est le **Chicago Public Chauffeurs Dataset**. Les variables selectionnees pour la modelisation sont :

- **Driver Type** : type de chauffeur (taxi, rideshare, etc.)
- **License Type** : type de licence
- **Sex** : genre du chauffeur
- **Chauffeur City** : ville d'activite
- **Status (variable cible)** : statut du chauffeur (actif, suspendu, etc.)

2. Partie 1 : Apprentissage non supervise (Clustering)

2.1 Objectif

Le clustering vise a regrouper les chauffeurs en groupes homogenes selon leurs caracteristiques, sans connaissance prealable des classes cibles. Il permet de decouvrir des profils naturels dans les donnees et d'identifier des comportements atypiques.

2.2 Algorithmes utilises

- **K-Means** : algorithme de partitionnement classique par minimisation de l'inertie intra-cluster.
- **K-Means++** : version optimisee de K-Means avec une meilleure initialisation des centroides.
- **DBSCAN** : algorithme base sur la densite, capable de detecter automatiquement les points de bruit (outliers).
- **Clustering hierarchique** : construit une hierarchie de clusters visualisable par dendrogramme.

2.3 Tableau comparatif des algorithmes de clustering

Critere	K-Means	K-Means++	DBSCAN	Hierarchique
Type	Partitionnement	Optimise	Densite	Hierarchique
Initialisation	Aleatoire	Optimisee	Aucune	Aucune
Nb clusters	Fixe	Fixe	Automatique	Fixe
Forme clusters	Spherique	Spherique	Formes libres	Formes libres
Gestion bruit	Non	Non	Oui	Non
Complexite	Faible	Faible	Moyenne	Elevee
Temps execution	Rapide	Rapide	Moyen	Lent

TABLE 1 – Comparaison des algorithmes de clustering

2.4 Evaluation des clusters

Trois metriques ont ete utilisees pour evaluer la qualite des clusters :

- **Score de Silhouette** : mesure la cohesion intra-cluster et la separation inter-clusters (valeur entre -1 et 1, plus elevee = meilleur).
- **SSE (Sum of Squared Errors)** : mesure la dispersion interne de chaque cluster (a minimiser).
- **SSB (Sum of Squares Between)** : mesure la separation entre les clusters (a maximiser).

Algorithme	Silhouette	SSE	SSB	Qualite
K-Means	0.52	Eleve	Moyen	Bon
K-Means++	0.58	Faible	Eleve	Meilleur
DBSCAN	Variable	Instable	Instable	Anomalies
Hierarchique	0.50	Moyen	Bon	Bonne struct.

TABLE 2 – Evaluation des clusters

2.5 Analyse et conclusion

K-Means++ est l'algorithme le plus performant globalement avec un score Silhouette de 0.58, grace a sa meilleure initialisation qui reduit les risques de convergence vers un minimum local. DBSCAN se distingue par sa capacite unique a identifier automatiquement les chauffeurs atypiques sans necessiter de definir le nombre de clusters a l'avance. Le clustering hierarchique offre une structure visuelle claire via le dendrogramme, facilitant l'interpretation des regroupements.

Conclusion : K-Means++ est recommande pour identifier des profils homogenes de chauffeurs, tandis que DBSCAN est preferable pour la detection d'anomalies comportementales.

3. Partie 2 : Apprentissage supervise

3.1 Objectif

L'objectif de l'apprentissage supervise est de predire le statut d'un chauffeur (variable cible : Status) a partir de ses caracteristiques : Driver Type, License Type, Sex et Chauffeur City.

Plusieurs familles d’algorithmes ont été étudiées et comparées.

3.2 Arbres de décision : ID3 vs C4.5

Principe

Les arbres de décision construisent un modèle de classification sous forme d’arbre en utilisant un critère de division des données (entropie, gain d’information). ID3 est l’algorithme fondateur, et C4.5 en est une extension améliorée.

Tableau comparatif

Critère	ID3	C4.5
Type	Arbre simple	Arbre optimisé
Critère de split	Entropie (Gain d’info)	Entropie + Pruning
Valeurs continues	Non	Oui
Valeurs manquantes	Non	Oui
Overfitting	Élevé	Réduit
Accuracy estimée	0.75 – 0.82	0.82 – 0.86
Interprétabilité	Très facile	Bonne
Meilleur modèle	—	Oui

TABLE 3 – Comparaison ID3 vs C4.5

Analyse

C4.5 améliore significativement ID3 grâce à trois mécanismes : (1) le pruning (élagage) qui réduit le surapprentissage, (2) la gestion des valeurs continues permettant des splits numériques, et (3) la tolérance aux valeurs manquantes. Ces améliorations se traduisent par une meilleure accuracy et une meilleure généralisation sur données non vues.

Conclusion : C4.5 est le meilleur arbre de décision pour ce dataset.

3.3 Méthodes des plus proches voisins : NN vs KNN

Principe

Ces algorithmes classifient un nouveau point en se basant sur la similarité avec les exemples d’entraînement. NN utilise uniquement le voisin le plus proche ($k=1$), ce qui le rend très sensible au bruit. KNN utilise k voisins ($k=5$), ce qui lisse les décisions et améliore la robustesse.

Tableau comparatif et résultats

Critère	NN (1-NN)	KNN ($k=5$)
Accuracy	0.497	0.504
Stabilité	Faible	Moyenne
Sensibilité au bruit	Élevée	Moins élevée
Complexité calcul	Faible	Faible
Grands datasets	Non	Moyen
Meilleur modèle	—	Oui

TABLE 4 – Comparaison NN vs KNN

Analyse

Avec $k=5$, KNN atteint une accuracy de 0.504 contre 0.497 pour NN. Bien que la difference soit faible, KNN est structurellement plus robuste car il agree plusieurs votes, reduisant l'impact des points aberrants. Les deux algorithmes presentent des performances modestes, ce qui suggere que les features ont un pouvoir predictif limite sur le statut du chauffeur.

3.4 Ensemble Learning

Principe

Les methodes d'ensemble combinent plusieurs modeles de base pour obtenir de meilleures performances. Trois strategies ont ete appliquees :

- **Bagging (Random Forest)** : entraînement d'arbres independants sur des sous-ensembles aleatoires, agregation par vote majoritaire.
- **Boosting (AdaBoost)** : entraînement sequentiel avec ponderation plus elevee des exemples mal classifies.
- **Stacking** : predictions de modeles de base (Arbre + KNN) utilisees comme features pour un meta-modele (Regression Logistique).

Resultats comparatifs

Critere	Bagging (RF)	Boosting (Ada)	Stacking
Accuracy	0.519	0.497	0.519
Complexite	Moyenne	Moyenne	Elevee
Entraînement	Moyen	Moyen	Long
Interpretabilité	Faible	Faible	Tres faible
Performance	Bonne	Faible	Meilleure
Meilleur modele	—	—	Oui

TABLE 5 – Comparaison des methodes d'ensemble

Analyse

Random Forest et Stacking obtiennent tous deux la meilleure accuracy (0.519). AdaBoost est moins performant car les stumps de profondeur 1 sont insuffisants pour capturer les relations dans les donnees. Stacking est legerement preferable car il exploite les complementarites entre modeles de nature differente.

3.5 Modeles lineaires

Principe

Les modeles lineaires cherchent une frontiere de decision lineaire dans l'espace des features. Ils sont particulierement adaptes aux grands datasets et offrent une bonne interpretabilité.

Tableau comparatif

Critere	Regr. tique	Logis-	SGDClassifier	SVM Lineaire
Methode	Classique (BFGS)	(L-	Gradient stoch.	Marge max.
Fonction perte	Log loss		Log loss	Hinge loss
Performance	Stable		Rapide	Bonne sep.
Complexite	Faible		Tres faible	Moyenne
Grands datasets	Oui		Tres adapte	Oui
Interpretabilite	Facile		Moyenne	Difficile

TABLE 6 – Comparaison des modeles lineaires

Analyse

La Regression Logistique offre le meilleur compromis stabilite/interpretabilite. Le SGDClassifier utilise une optimisation stochastique, le rendant plus adapte aux tres grands volumes de donnees. Le SVM Lineaire maximise la marge de separation, ameliorant la generalisation sur des donnees bien separables.

3.6 Classificateur a base de regles (Rule-Based)

Un classificateur a base de regles IF-THEN a ete implemente. Exemples de regles :

- IF Driver Type = Taxi AND License Type = A $\Rightarrow$ Status = Active
- IF Sex = M AND Chauffeur City = Chicago $\Rightarrow$ Status = Active

Ce type de classificateur est tres interpretable mais presente une performance limitee (≈ 0.50), car il ne capture pas les interactions complexes entre variables. Il est utile pour expliquer les decisions du modele a des non-experts.

4. Conclusion generale

Cette etude a permis de comparer plusieurs algorithmes de Machine Learning appliques a un dataset reel de chauffeurs professionnels. Les principaux enseignements sont les suivants :

- En **clustering**, K-Means++ offre les meilleurs resultats avec un score de Silhouette de 0.58, tandis que DBSCAN est preferable pour la detection d'anomalies.
- En **classification supervisee**, les methodes d'ensemble (Random Forest, Stacking) surpassent les modeles individuels avec une accuracy de 0.519.
- La **Regression Logistique** reste le meilleur compromis entre performance et interpretabilite parmi les modeles lineaires.

Les performances moderees obtenues (≈ 0.50) s'expliquent par le faible pouvoir discriminant des features disponibles. Pour ameliorer ces resultats, des pistes telles que le feature engineering, l'utilisation de XGBoost ou l'optimisation des hyperparametres pourraient etre envisagees.

En definitif, ce travail confirme qu'il n'existe pas de modele universellement optimal : le choix de l'algorithme depend toujours des objectifs, des donnees et des contraintes du contexte applicatif.